

Exercise 1: Control Structures

Scenario 1: Apply loan discount for seniors

```
BEGIN
  FOR r_loan IN (SELECT l.LoanID, l.InterestRate
                 FROM Loans l
                 JOIN Customers c ON l.CustomerID = c.CustomerID
                 WHERE MONTHS_BETWEEN(SYSDATE, c.DOB) / 12 > 60)
  LOOP
    UPDATE Loans
    SET InterestRate = InterestRate - 1
    WHERE LoanID = r_loan.LoanID;
  END LOOP;
  COMMIT;
END;
/
```

Scenario 2: Promote VIP status based on balance

```
BEGIN
  FOR r_cust IN (SELECT CustomerID FROM Customers WHERE Balance > 10000)
  LOOP
    -- Assuming an IsVIP column exists or updating placeholder logic
    UPDATE Customers
    SET IsVIP = 'TRUE' -- or 1 depending on column data type
    WHERE CustomerID = r_cust.CustomerID;
  END LOOP;
  COMMIT;
END;
/
```

Scenario 3: Send loan due reminders (Next 30 Days)

```
SET SERVEROUTPUT ON;
BEGIN
  FOR r_loan IN (SELECT l.LoanID, c.Name, l.EndDate
                 FROM Loans l
                 JOIN Customers c ON l.CustomerID = c.CustomerID
                 WHERE l.EndDate BETWEEN SYSDATE AND SYSDATE + 30)
  LOOP
    DBMS_OUTPUT.PUT_LINE('Reminder: Customer ' || r_loan.Name ||
                          ' has Loan ID ' || r_loan.LoanID ||
                          ' due on ' || TO_CHAR(r_loan.EndDate, 'YYYY-MM-DD') || '.');
  END LOOP;
END;
/
```

Exercise 2: Error Handling

Scenario 1: Safe fund transfer

```
CREATE OR REPLACE PROCEDURE SafeTransferFunds (  
    p_from_acc IN NUMBER,  
    p_to_acc  IN NUMBER,  
    p_amount  IN NUMBER  
) AS  
    v_balance NUMBER;  
    v_insufficient_funds EXCEPTION;  
BEGIN  
    -- Check balance  
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = p_from_acc;  
  
    IF v_balance < p_amount THEN  
        RAISE v_insufficient_funds;  
    END IF;  
  
    -- Deduct funds  
    UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID =  
p_from_acc;  
    -- Add funds  
    UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID = p_to_acc;  
  
    COMMIT;  
    DBMS_OUTPUT.PUT_LINE('Transfer successful.');
```

EXCEPTION

```
    WHEN v_insufficient_funds THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: Insufficient funds in account ' || p_from_acc);  
    WHEN NO_DATA_FOUND THEN  
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error: One or both Account IDs do not exist.');
```

WHEN OTHERS THEN

```
        ROLLBACK;  
        DBMS_OUTPUT.PUT_LINE('Error occurred: ' || SQLERRM);  
END;  
/
```

Scenario 2: Manage employee salary updates

```
CREATE OR REPLACE PROCEDURE UpdateSalary (  
    p_emp_id  IN NUMBER,  
    p_percentage IN NUMBER  
) AS
```

```

BEGIN
    UPDATE Employees
    SET Salary = Salary + (Salary * (p_percentage / 100))
    WHERE EmployeeID = p_emp_id;

    IF SQL%ROWCOUNT = 0 THEN
        RAISE NO_DATA_FOUND;
    END IF;

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Salary updated successfully.');
```

EXCEPTION

```

    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Error: Employee ID ' || p_emp_id || ' does not exist.');
```

WHEN OTHERS THEN

```

    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

Scenario 3: Prevent duplicate customer insertion

```

CREATE OR REPLACE PROCEDURE AddNewCustomer (
    p_cust_id IN NUMBER,
    p_name    IN VARCHAR2,
    p_dob     IN DATE,
    p_balance IN NUMBER
) AS
BEGIN
    INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
    VALUES (p_cust_id, p_name, p_dob, p_balance, SYSDATE);

    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Customer added successfully.');
```

EXCEPTION

```

    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Error: Customer with ID ' || p_cust_id || ' already exists.
Insertion aborted.');
```

WHEN OTHERS THEN

```

    DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
END;
/
```

Exercise 3: Stored Procedures

Scenario 1: Process monthly interest

```

CREATE OR REPLACE PROCEDURE ProcessMonthlyInterest AS
BEGIN
```

```

UPDATE Accounts
SET Balance = Balance * 1.01
WHERE AccountType = 'Savings';

COMMIT;
DBMS_OUTPUT.PUT_LINE('Monthly interest applied to all savings accounts.');
```

END;

/

Scenario 2: Update employee bonuses by department

```

CREATE OR REPLACE PROCEDURE UpdateEmployeeBonus (
  p_dept    IN VARCHAR2,
  p_bonus_pct IN NUMBER
) AS
BEGIN
  UPDATE Employees
  SET Salary = Salary + (Salary * (p_bonus_pct / 100))
  WHERE Department = p_dept;

  COMMIT;
  DBMS_OUTPUT.PUT_LINE('Bonuses applied to department: ' || p_dept);
```

END;

/

Scenario 3: Standard Fund Transfer

```

CREATE OR REPLACE PROCEDURE TransferFunds (
  p_src_acc IN NUMBER,
  p_dest_acc IN NUMBER,
  p_amount  IN NUMBER
) AS
  v_src_bal NUMBER;
BEGIN
  SELECT Balance INTO v_src_bal FROM Accounts WHERE AccountID = p_src_acc;

  IF v_src_bal >= p_amount THEN
    UPDATE Accounts SET Balance = Balance - p_amount WHERE AccountID =
p_src_acc;
    UPDATE Accounts SET Balance = Balance + p_amount WHERE AccountID =
p_dest_acc;
    COMMIT;
    DBMS_OUTPUT.PUT_LINE('Successfully transferred ' || p_amount);
  ELSE
    DBMS_OUTPUT.PUT_LINE('Transfer Failed: Insufficient Balance.');
```

END IF;

END;

/

Exercise 4: Functions

Scenario 1: Calculate age

```
CREATE OR REPLACE FUNCTION CalculateAge (  
    p_dob IN DATE  
) RETURN NUMBER AS  
BEGIN  
    RETURN FLOOR(MONTHS_BETWEEN(SYSDATE, p_dob) / 12);  
END;  
/
```

Scenario 2: Compute monthly loan installment

```
CREATE OR REPLACE FUNCTION CalculateMonthlyInstallment (  
    p_loan_amount IN NUMBER,  
    p_interest_rate IN NUMBER, -- Annual percentage rate (e.g., 5 for 5%)  
    p_duration_years IN NUMBER  
) RETURN NUMBER AS  
    v_monthly_rate NUMBER;  
    v_total_months NUMBER;  
    v_emi NUMBER;  
BEGIN  
    v_monthly_rate := (p_interest_rate / 100) / 12;  
    v_total_months := p_duration_years * 12;  
  
    -- Formula: EMI = [P x R x (1+R)^N]/[(1+R)^N-1]  
    IF v_monthly_rate = 0 THEN  
        v_emi := p_loan_amount / v_total_months;  
    ELSE  
        v_emi := (p_loan_amount * v_monthly_rate * POWER(1 + v_monthly_rate,  
v_total_months)) /  
            (POWER(1 + v_monthly_rate, v_total_months) - 1);  
    END IF;  
  
    RETURN ROUND(v_emi, 2);  
END;  
/
```

Scenario 3: Check for sufficient balance

```
CREATE OR REPLACE FUNCTION HasSufficientBalance (  
    p_account_id IN NUMBER,  
    p_amount IN NUMBER  
) RETURN BOOLEAN AS  
    v_balance NUMBER;  
BEGIN  
    SELECT Balance INTO v_balance FROM Accounts WHERE AccountID = p_account_id;  
    IF v_balance >= p_amount THEN
```

```

        RETURN TRUE;
    ELSE
        RETURN FALSE;
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN FALSE;
END;
/

```

Exercise 5: Triggers

Scenario 1: Auto-update last modified timestamp

```

CREATE OR REPLACE TRIGGER UpdateCustomerLastModified
BEFORE UPDATE ON Customers
FOR EACH ROW
BEGIN
    :NEW.LastModified := SYSDATE;
END;
/

```

Scenario 2: Transaction Audit Log

-- Supporting table setup for the script

```

CREATE TABLE AuditLog (
    LogID NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    TransactionID NUMBER,
    AccountID NUMBER,
    LogDate DATE,
    Amount NUMBER,
    Type VARCHAR2(10)
);

```

```

CREATE OR REPLACE TRIGGER LogTransaction
AFTER INSERT ON Transactions
FOR EACH ROW
BEGIN
    INSERT INTO AuditLog (TransactionID, AccountID, LogDate, Amount, Type)
    VALUES (:NEW.TransactionID, :NEW.AccountID, SYSDATE, :NEW.Amount,
:NEW.TransactionType);
END;
/

```

Scenario 3: Enforce business rules on transactions

```

CREATE OR REPLACE TRIGGER CheckTransactionRules
BEFORE INSERT ON Transactions
FOR EACH ROW
DECLARE

```

```

    v_current_bal NUMBER;
BEGIN
    -- Enforce positive deposits
    IF :NEW.TransactionType = 'Deposit' AND :NEW.Amount <= 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Deposit amounts must be greater than zero.');
```

END IF;

```

    -- Enforce available balance for withdrawals
    IF :NEW.TransactionType = 'Withdrawal' THEN
        SELECT Balance INTO v_current_bal FROM Accounts WHERE AccountID =
:NEW.AccountID;
        IF :NEW.Amount > v_current_bal THEN
            RAISE_APPLICATION_ERROR(-20002, 'Withdrawal rejected: Insufficient account
balance.');
```

END IF;

```

    END IF;
END;
/
```

Exercise 6: Cursors

Scenario 1: Generate monthly statements

```

DECLARE
    CURSOR c_monthly_statements IS
        SELECT c.Name, t.AccountID, t.TransactionDate, t.Amount, t.TransactionType
        FROM Transactions t
        JOIN Accounts a ON t.AccountID = a.AccountID
        JOIN Customers c ON a.CustomerID = c.CustomerID
        WHERE EXTRACT(MONTH FROM t.TransactionDate) = EXTRACT(MONTH FROM
SYSDATE)
        AND EXTRACT(YEAR FROM t.TransactionDate) = EXTRACT(YEAR FROM
SYSDATE);

    r_stmt c_monthly_statements%ROWTYPE;
BEGIN
    OPEN c_monthly_statements;
    LOOP
        FETCH c_monthly_statements INTO r_stmt;
        EXIT WHEN c_monthly_statements%NOTFOUND;

        DBMS_OUTPUT.PUT_LINE('Customer: ' || r_stmt.Name ||
            ' | Acc: ' || r_stmt.AccountID ||
            ' | Date: ' || TO_CHAR(r_stmt.TransactionDate, 'YYYY-MM-DD') ||
            ' | Type: ' || r_stmt.TransactionType ||
            ' | Amount: $' || r_stmt.Amount);
    END LOOP;
    CLOSE c_monthly_statements;
END;
/
```

Scenario 2: Deduct annual maintenance fee

```
DECLARE
  v_fee CONSTANT NUMBER := 50; -- Example fee value
  CURSOR c_accounts IS
    SELECT AccountID FROM Accounts FOR UPDATE;
BEGIN
  FOR r_acc IN c_accounts LOOP
    UPDATE Accounts
      SET Balance = Balance - v_fee,
          LastModified = SYSDATE
    WHERE CURRENT OF c_accounts;
  END LOOP;
  COMMIT;
END;
/
```

Scenario 3: Update interest rate based on policy

```
DECLARE
  CURSOR c_loans IS
    SELECT LoanID, LoanAmount FROM Loans FOR UPDATE;
  v_new_rate NUMBER;
BEGIN
  FOR r_loan IN c_loans LOOP
    -- Example Rule Policy: Loans over $10k get a lower rate (4%), others get 6%
    IF r_loan.LoanAmount > 10000 THEN
      v_new_rate := 4;
    ELSE
      v_new_rate := 6;
    END IF;

    UPDATE Loans
      SET InterestRate = v_new_rate
    WHERE CURRENT OF c_loans;
  END LOOP;
  COMMIT;
END;
/
```

Exercise 7: Packages

Scenario 1: Customer Management Package

-- Specification

```
CREATE OR REPLACE PACKAGE CustomerManagement AS
  PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal
NUMBER);
  PROCEDURE UpdateDetails(p_id NUMBER, p_name VARCHAR2);
```



```

    FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER;
END CustomerManagement;
/

-- Body
CREATE OR REPLACE PACKAGE BODY CustomerManagement AS
    PROCEDURE AddCustomer(p_id NUMBER, p_name VARCHAR2, p_dob DATE, p_bal
NUMBER) AS
    BEGIN
        INSERT INTO Customers (CustomerID, Name, DOB, Balance, LastModified)
        VALUES (p_id, p_name, p_dob, p_bal, SYSDATE);
    END AddCustomer;

    PROCEDURE UpdateDetails(p_id NUMBER, p_name VARCHAR2) AS
    BEGIN
        UPDATE Customers SET Name = p_name WHERE CustomerID = p_id;
    END UpdateDetails;

    FUNCTION GetBalance(p_id NUMBER) RETURN NUMBER AS
        v_bal NUMBER;
    BEGIN
        SELECT Balance INTO v_bal FROM Customers WHERE CustomerID = p_id;
        RETURN v_bal;
    END GetBalance;
END CustomerManagement;
/

```

Scenario 2: Employee Management Package

```

-- Specification
CREATE OR REPLACE PACKAGE EmployeeManagement AS
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_pos VARCHAR2,
p_sal NUMBER, p_dept VARCHAR2);
    PROCEDURE UpdateDetails(p_id NUMBER, p_pos VARCHAR2, p_sal NUMBER);
    FUNCTION GetAnnualSalary(p_id NUMBER) RETURN NUMBER;
END EmployeeManagement;
/

-- Body
CREATE OR REPLACE PACKAGE BODY EmployeeManagement AS
    PROCEDURE HireEmployee(p_id NUMBER, p_name VARCHAR2, p_pos VARCHAR2,
p_sal NUMBER, p_dept VARCHAR2) AS
    BEGIN
        INSERT INTO Employees (EmployeeID, Name, Position, Salary, Department,
HireDate)
        VALUES (p_id, p_name, p_pos, p_sal, p_dept, SYSDATE);
    END HireEmployee;

```

```

PROCEDURE UpdateDetails(p_id NUMBER, p_pos VARCHAR2, p_sal NUMBER) AS
BEGIN
    UPDATE Employees SET Position = p_pos, Salary = p_sal WHERE EmployeeID =
p_id;
END UpdateDetails;

FUNCTION GetAnnualSalary(p_id NUMBER) RETURN NUMBER AS
    v_sal NUMBER;
BEGIN
    SELECT Salary INTO v_sal FROM Employees WHERE EmployeeID = p_id;
    RETURN v_sal * 12;
END GetAnnualSalary;
END EmployeeManagement;
/

```

Scenario 3: Account Operations Package

-- Specification

```

CREATE OR REPLACE PACKAGE AccountOperations AS
    PROCEDURE OpenAccount(p_acc_id NUMBER, p_cust_id NUMBER, p_type
VARCHAR2, p_bal NUMBER);
    PROCEDURE CloseAccount(p_acc_id NUMBER);
    FUNCTION GetTotalBalance(p_cust_id NUMBER) RETURN NUMBER;
END AccountOperations;
/

```

-- Body

```

CREATE OR REPLACE PACKAGE BODY AccountOperations AS
    PROCEDURE OpenAccount(p_acc_id NUMBER, p_cust_id NUMBER, p_type
VARCHAR2, p_bal NUMBER) AS
    BEGIN
        INSERT INTO Accounts (AccountID, CustomerID, AccountType, Balance,
LastModified)
        VALUES (p_acc_id, p_cust_id, p_type, p_bal, SYSDATE);
    END OpenAccount;

    PROCEDURE CloseAccount(p_acc_id NUMBER) AS
    BEGIN
        DELETE FROM Accounts WHERE AccountID = p_acc_id;
    END CloseAccount;

    FUNCTION GetTotalBalance(p_cust_id NUMBER) RETURN NUMBER AS
        v_total NUMBER;
    BEGIN
        SELECT SUM(Balance) INTO v_total FROM Accounts WHERE CustomerID =
p_cust_id;
        RETURN NVL(v_total, 0);
    END GetTotalBalance;

```

END AccountOperations;
/